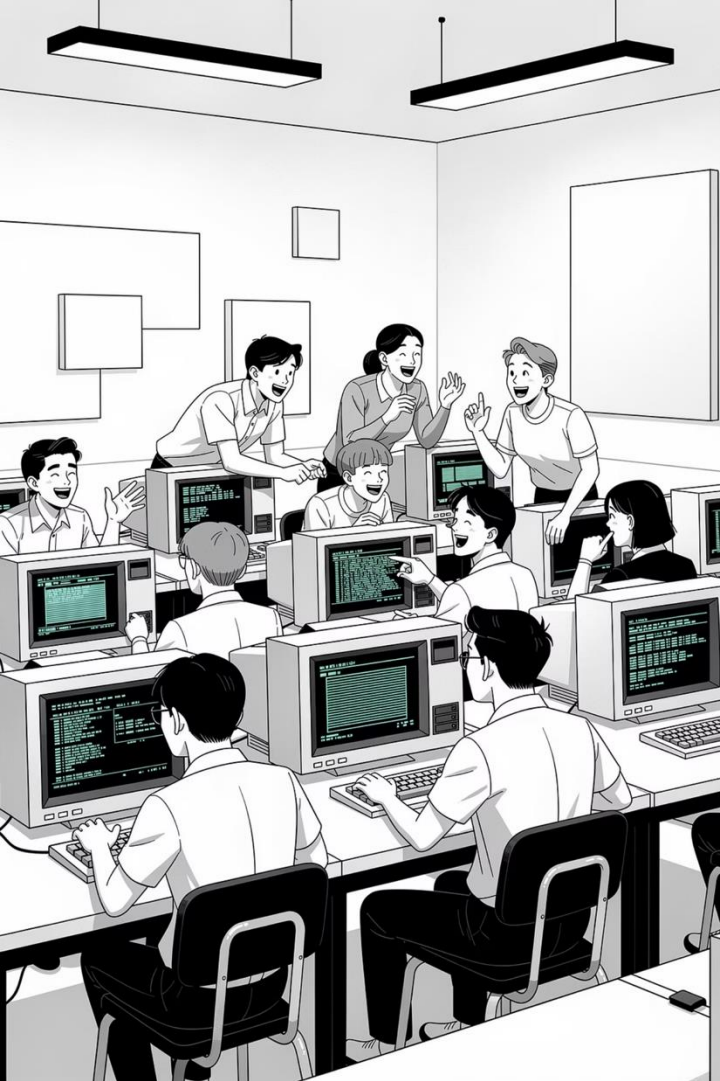




# Software Design

A journey through ObjectSpeak, object frenzy, and other delightful nonsense — plus the real story behind OOP, quality, and why your code keeps breaking at 2am.

What We'll Cover

# Today's Agenda

01

---

ObjectSpeak & Buzzwords

The hype, the nonsense, and the greatest lie  
in tech

02

---

What OOP Actually Is

Spoiler: it's about classes, not objects!

03

---

Quality vs. Productivity

Why doing it right beats doing it over

04

---

The Key Elements

Reusability, Extendibility, and more

05

---

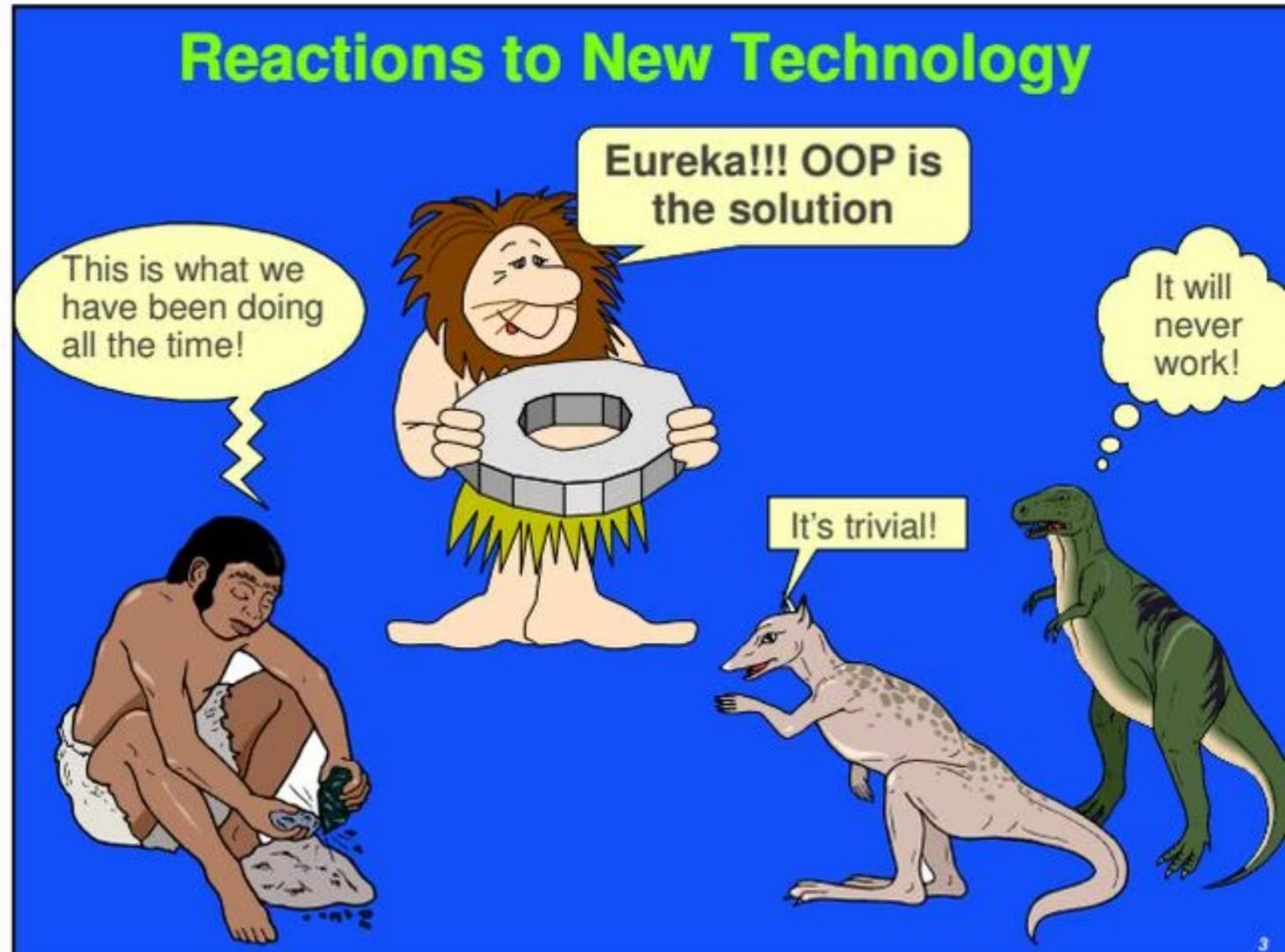
Limits & Caveats

OOP is not magic — sorry!

# Reactions to New Technology

Every generation, every paradigm shift:  
same four characters show up.

Which one are you? 🐾



## Chapter 1

# ObjectSpeak & Other Nonsense

### "ObjectSpeak" Hall of Fame

- "An object-oriented framework that monitors UNIX systems."
- "Hybrid object/relational database."
- "CA-Visual Objects for Windows."
- Microsoft's "Object Linking and Embedding."

### Other Nonsense

- "Windows-NT is Object Oriented!"
- "ADA is an OO Language."
- "Experience required in OOP using C or C++."
- M.Sc. thesis: "Object-Oriented, Reflective, Database..."

### The Greatest Lie 🤨

**OOP is about Objects!** (It's not. Stay tuned.)

# Object Frenzy: A 1982 Prophecy

"Object oriented programming will be in the 1980s what structured programming was in the 1970s: Everyone will be in favour of it. Every manufacturer will promote his products as supporting it. Every manager will pay lip service to it. Every programmer will practice it (differently). And, no one will know just what it is."

— T. Rentsch, 1982 🤖

📄 Remarkably accurate. Rentsch basically predicted every tech trend ever.

# OOP: Also Known As...

 An Amusing Acronym

Yes, OOP. We all noticed.

 Synonym for Slow Graphics

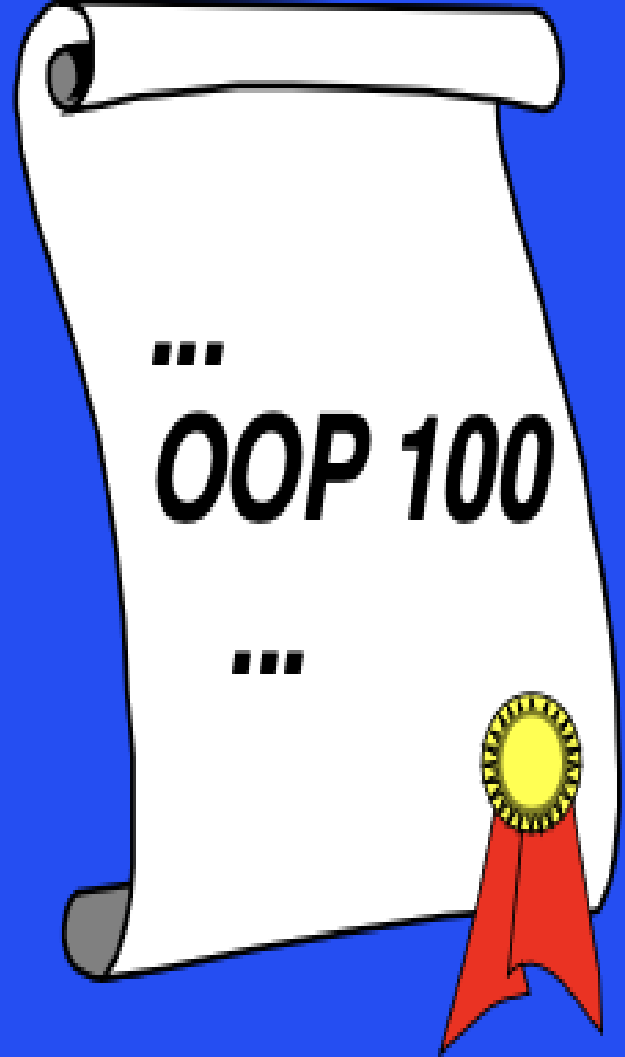
Early OOP had... performance issues.

 Resume Buzzword

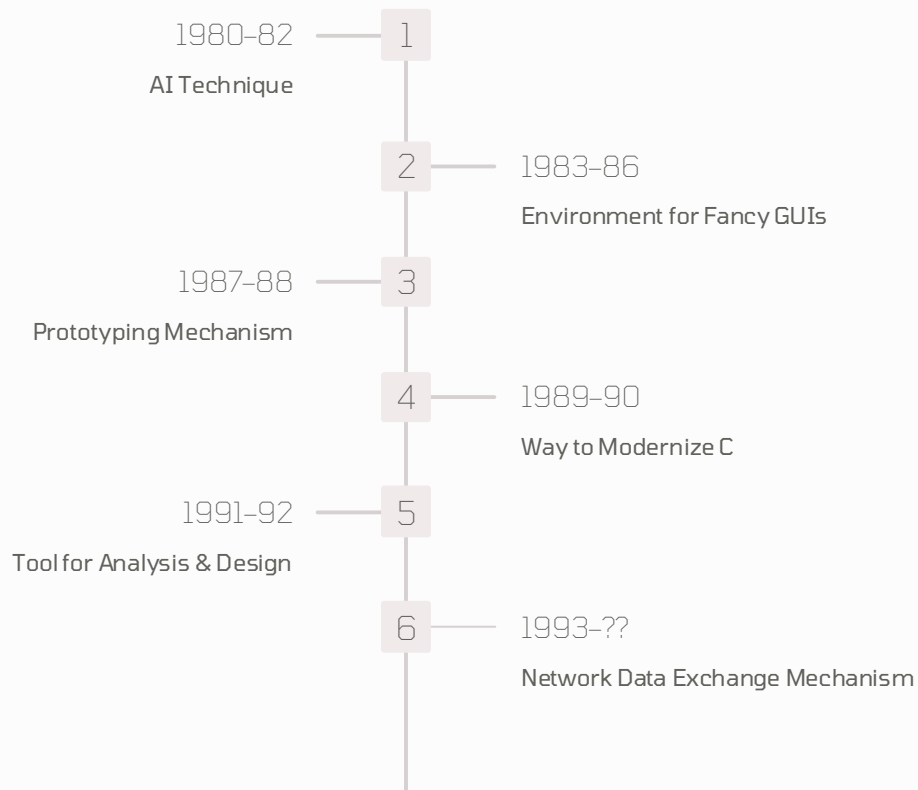
A valuable addition to any CV, regardless of understanding.

 Many Textbooks

"OOP == OOP in X!" where X = the author's favourite language.



# OOP's Identity Crisis Through the Decades



OOP has been everything to everyone. A true chameleon of computer science. 🦎



## The Syllogism Problem

*X is good.*

*Object oriented is good.*

*Therefore, X is object oriented.*

Where X = Ada, CorelDraw, Roger King's Cat, Visual Basic, Microsoft Excel, Windows-NT, communication programs...

⚠️ Yes, someone's cat was declared object-oriented. This is not a drill. 🐱

## Chapter 2

# What OOP Actually Is

Primarily: A Software Engineering Technique

Methods of abstraction:

- **Class** (Most Important!)
- Inheritance
- Genericity

**OOP is about classes, not about objects!**

Deliverables

- **Reliability:** Bug-free software
- **Extendibility:** Adapts to changing needs
- **Reusability:** Build from existing parts
- **Portability:** Runs on many hardware systems
- **Efficiency:** High-performance software

# Mission: Quality Software!

*"When quality is pursued, productivity will follow!"*

— K. Fujino, Vice President of NEC Corporation of Japan



# What About Productivity?

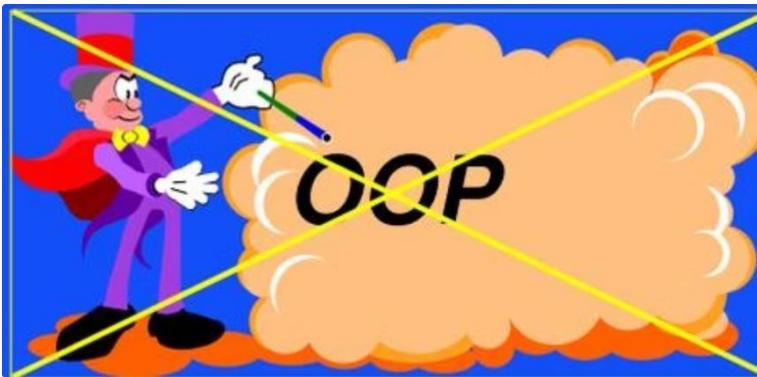
*"There is never time to do it right, but there is always time to do it over."*



⚠ Sound familiar? Quality first saves time. Cutting corners just reschedules the pain. 😬

## Chapter 3

# Caveats: OOP Is Not a Magic Wand



### OOP Does Not Address Everything

It will not turn bad programmers into good ones. Sorry, no shortcuts here. 🙄

### No Guarantees Either

Even when OOP addresses an issue, it's not guaranteed to solve it. Software construction is tough — do not expect miracles.

# The Limits of OOP

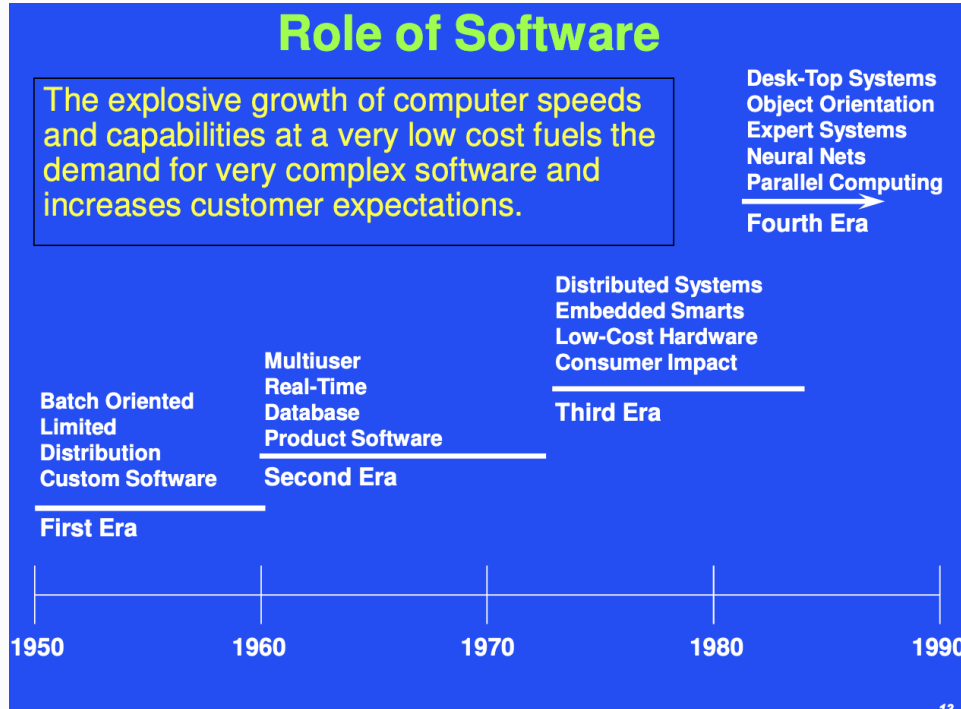
## The Dream: Software Like Light Bulbs

- Reduce production time
- Reduce production cost
- Detect failures before delivery
- Establish good policies for fixing failures
- Measure production progress

## The Reality

OOP helps, but does not solve the problems. Software is like mathematical theorems — good notations and tools help, but the main factor remains: **human Ingenuity**. 🧠

# The Role of Software Through History



The explosive growth of computer speeds and capabilities at very low cost fuels demand for complex software and raises customer expectations — from batch-oriented custom software in the 1950s all the way to Object Orientation, Expert Systems, and Neural Nets in the Fourth Era.

# The Software Crisis: By the Numbers

\$750K

256K Computer (1965)

vs. just \$50 in 1995

\$50K

Programmer/yr (1995)

vs. \$2,500 in 1965

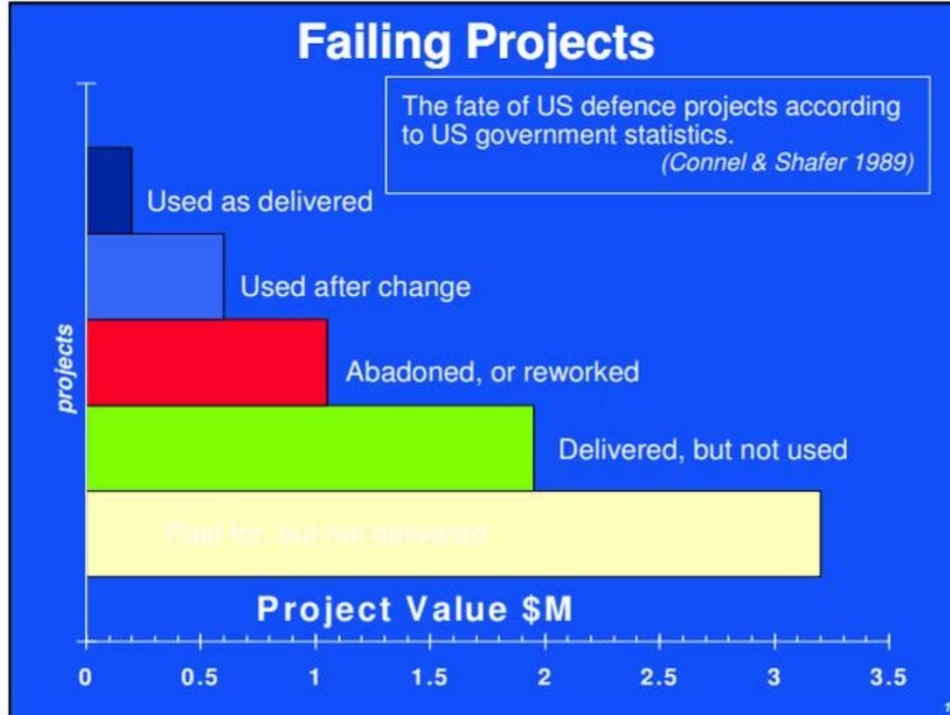
\$1K

4MB Computer (1995)

Hardware costs plummeted; software costs soared

⊗ Software projects are: **Late! Buggy! Inflexible!** — The amount spent on software increases with time. Hardware? Decreases. 📈

# Failing Projects: The Uncomfortable Truth



The fate of US defence projects (Connel & Shafer 1989). Only a tiny fraction was used as delivered. The rest? Abandoned, reworked, or paid for but never delivered. 🗑️

# Simple vs. Complex Software

## Simple Software System

Small set of behaviours. Understandable by a single person.

Trashable — if it breaks, just throw it away!

## Complex Software System

Rich set of behaviours. Exceeds average human intellectual capacity. Cannot afford to throw away — must patch, maintain, and carefully control changes.

The answer to complexity? **Reusability, Extendibility, Compatibility.**

## Chapter 5

# Why Does Software Fail? Reason #1

### Complexity & Haste

A skilled programmer writes **10–15 lines of code per day** at ~\$18/line. Can't afford that? You might hire non-professionals or cut corners. Neither ends well. 🤖

### How OOP Fights Complexity

- A more realistic model of the world: collection of interacting objects instead of a collection of functions
- Reuse of ready-made components

# Why Does Software Fail? Reason #2

## Interconnectedness

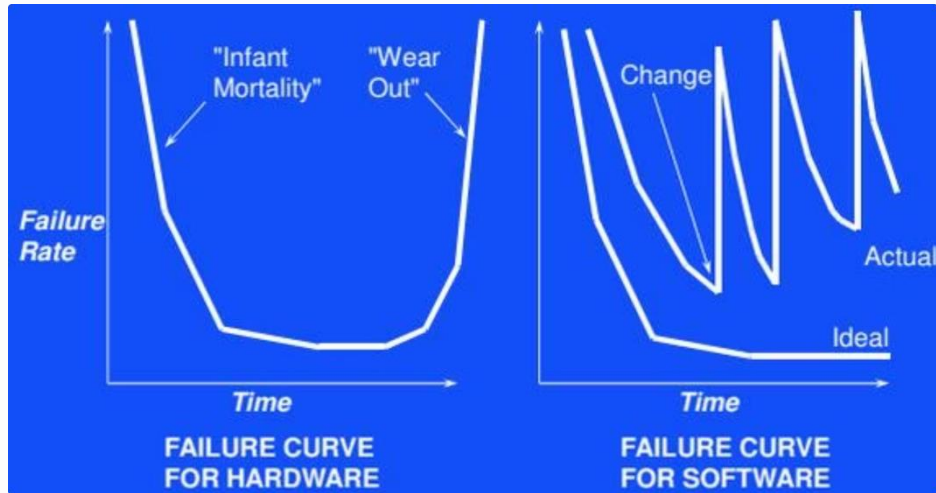
Too much interaction between team members leads to the "integration nightmare." Classic example: 1 child = 9 months, no matter how many women are assigned. Adding programmers *lengthens*, not shortens, the schedule.

## OOP & Interconnectedness

Warning: OOP, if not used carefully, can actually *increase* interconnectivity. Used well, it has the potential for more disciplined, manageable interconnectivity.

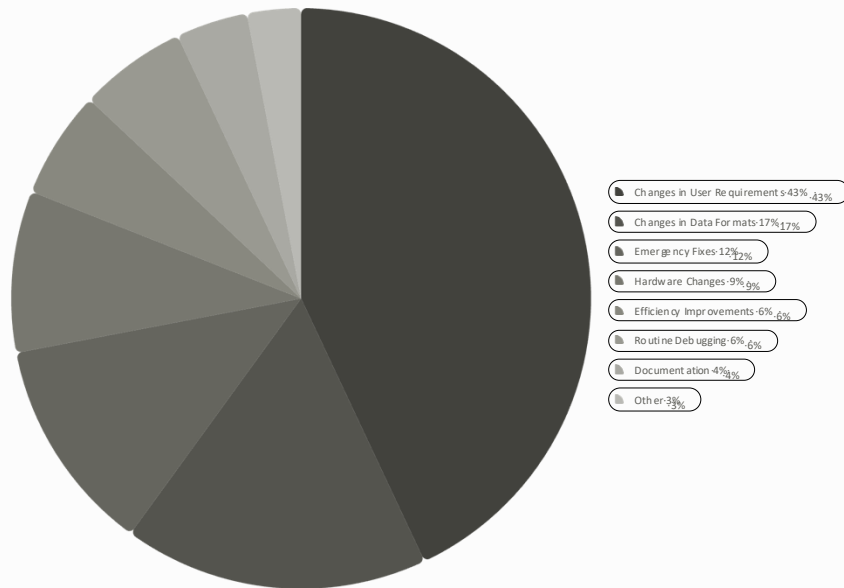
# Why Does Software Fail? Reason #3

Software is "soft" — it's constantly subject to conflicting demands, fluctuating requirements, and optimization pressures. Unlike hardware, it doesn't wear out. It just... changes. Constantly. 🔄



# Maintenance: 70% of Software Cost

Breakdown of Maintenance Costs — Lientz & Swanson '79



43% of maintenance cost comes from changes in user requirements. Users keep changing their minds. Who knew? 🤖

# OOP & Changes to User Requirements

## Characteristics of Requirements Changes

- Same entities
- Similar relationships (is-a / has-a) between entities
- Different interactions

Result: Very little change to the data model. Good news! 🎉

## OOP Promotes Extensibility

Because OOP models the world as stable entities with flexible interactions, it naturally accommodates changing requirements without requiring a complete rewrite.

# OOP vs. Traditional Paradigm

## Traditional (Function-Oriented)

**Verbs supported by Nouns.** Identify the algorithm (functions) first, then the data structures needed by those functions. Control flow drives everything.

## Object-Oriented

**Nouns supported by Verbs.** Identify abstract objects (data) representing the problem domain first. Then define the operations (interface) supported by those objects.

 The solution becomes a sequence of calls to objects — elegant, modular, and maintainable.

## Chapter 6

# OOP and Software Quality

OOP is, at its core, a **Software Engineering Technique** — and Software Engineering has one goal: **produce quality practical software.**

### Productivity

Getting things done fast

### Quality

Internal & external factors

### Practicality

Real-world constraints



# External Quality Factors: Product Economy



## Efficiency

Performance in time, memory, disk, bandwidth, processors. Accounts for ~4% of maintenance costs.



## Integrity

Protection against malice and accidents. Security matters more than ever.



## Usability

Ease of operation, learning, input/output, and error recovery. Happy users = happy life.

# External Quality Factors: Development Economy



## Extensibility

Adaptability to change — accounts for 43% of maintenance costs. The big one!



## Reusability

Use components in other applications. Build once, use many times.



## Compatibility

Works with other software components. Plays well with others.



## Portability

Runs on other software/hardware environments. Accounts for ~6% of maintenance costs.

# Verifiability: The Management Community's Concern

## Why Verifiability Matters

Ease of designing and applying test data and acceptance procedures.

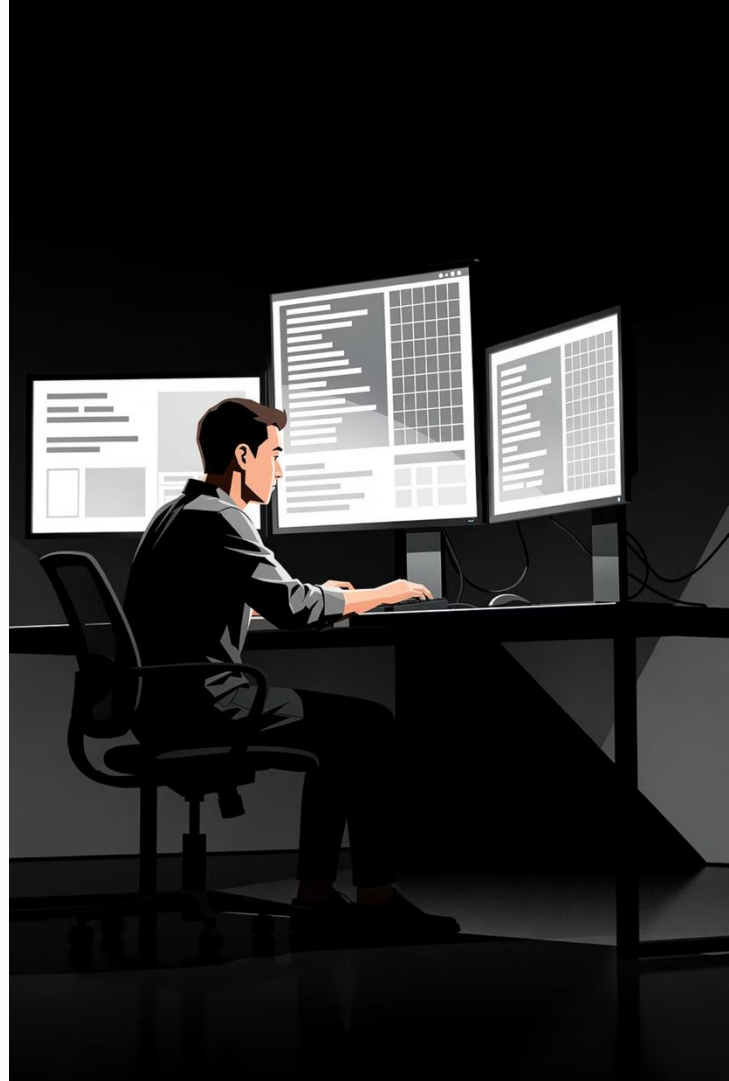
Bugs are expensive — especially the ones you find after shipping. 🐛

## The Numbers

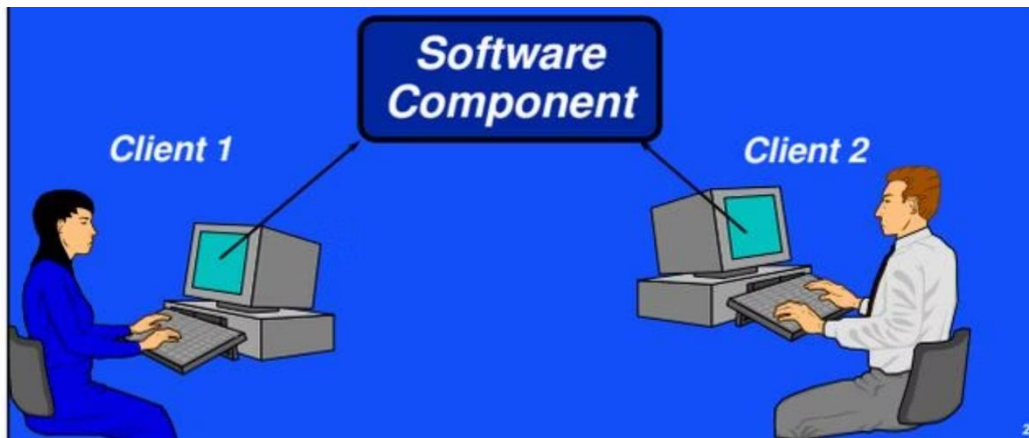
- **9%** of maintenance costs: Routine debugging
- **12%** of maintenance costs: Emergency fixes

*Many quality factors are correlated.*

*Many are conflicting!*



# Reusability



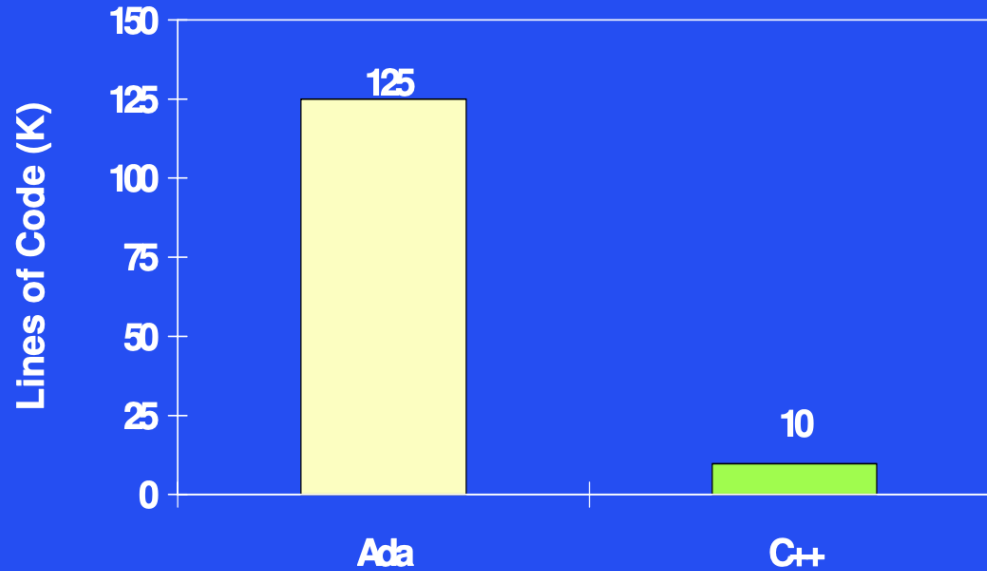
Reduce Development Cost

One component instead of many. Why build it twice?

Promote Reliability

Exercise the component from many different aspects. No need to remember to fix the same error twice. 🔄

## Code Reuse in C++

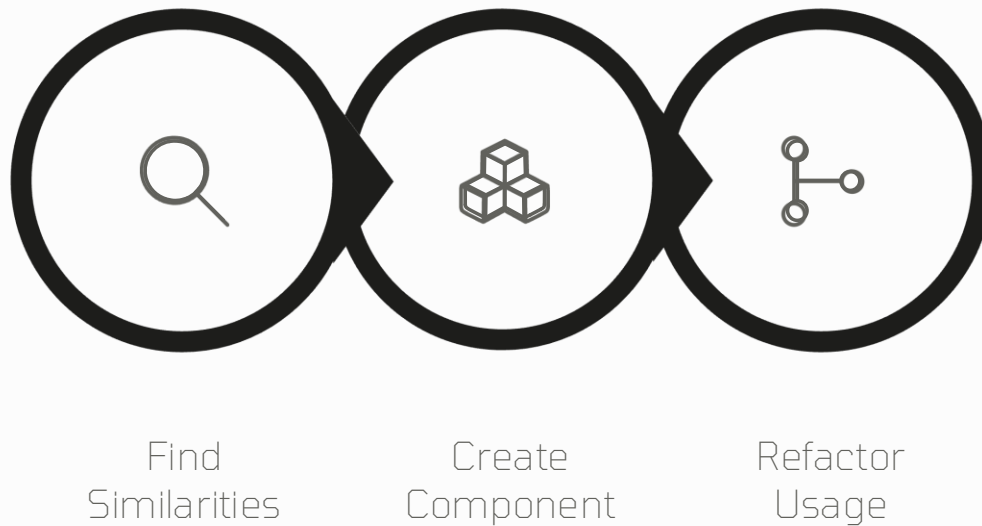


**The Power of Inheritance:** Reduction in size of the *Booch Components*, originally written in Ada as a result of the port to C++.

# Reuse, Reuse, Reuse!

*"The Clinton Administration has targeted 5 technology areas for support from the National Institute of Standards & Technology, and two focus directly on information technology: a nationwide health care information infrastructure and an effort to develop reusable software components for large software systems."*

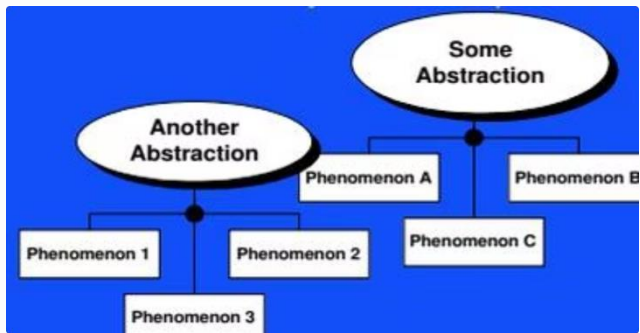
— Wall Street Journal, 26 April '94, Page B6



Reuse is tightly coupled with abstraction. Find the common parts, extract them, and share them everywhere.

## Chapter 8

# Abstraction: Finding the Commonality



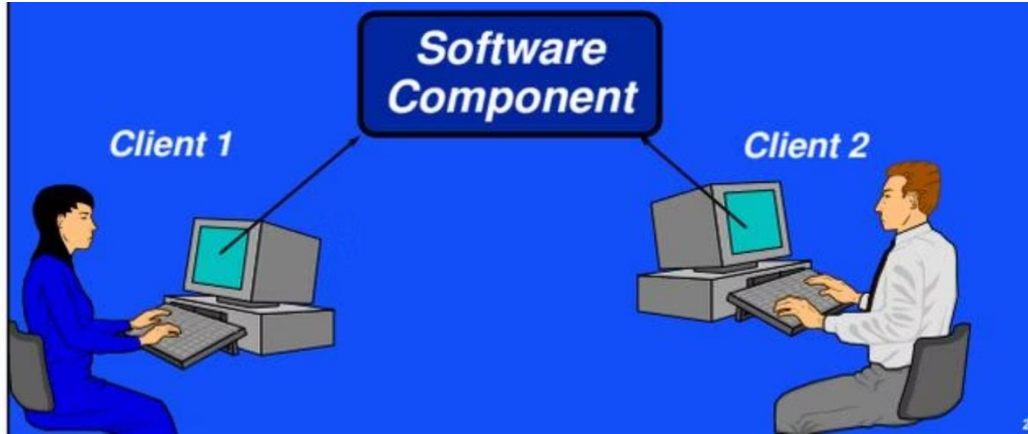
What is Abstraction?

Find commonality of several phenomena. Ignore the "irrelevant" details and maximize "relevant" commonalities.

Good Abstraction

Abstractions are almost invariably relative to a domain of interest. Context is everything!

# Abstraction & Reusability: The Connection



Reduce Development Cost

One component instead of many — build it once, use it everywhere.

Promote Reliability

Exercise the component from many different aspects. No need to fix the same error in multiple places. 🛠️

# The Principle of Little Numbers

## 0 > 1: Information Hiding

What you don't know cannot hurt you. Encapsulation, table-driven programming, modularity — hide the details!

## 1 > 2: No Duplication

If the same thing occurs twice in the code, it occurs one time too many. As changes occur, maintaining two almost identical parts becomes a nightmare. 🤖

```
For (ever) {  
  Write Code;  
  While (exist(similar parts)) {  
    Extract identical portion;  
    Rewrite code;  
  }  
}
```

# Redundancy is Redundant!

## Code

Procedures, functions, libraries, macros, software reusability, code sharing...

## Comments

"Don't repeat the code in your comments." (You know who you are.)

## Text

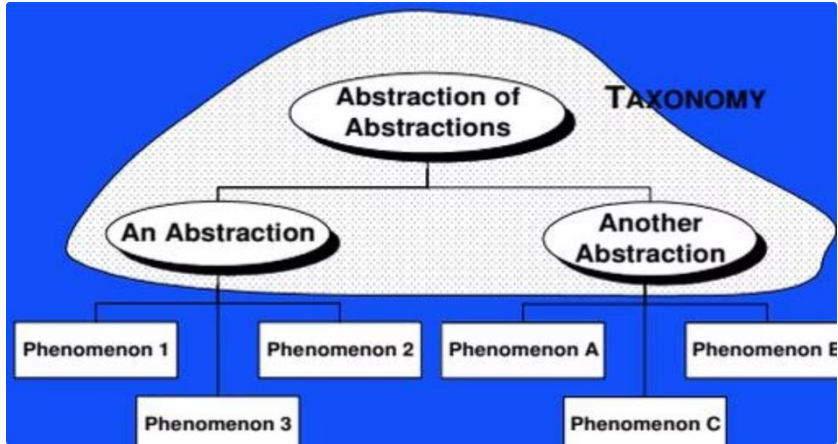
"Don't prepare the table of contents manually." Automate it!

## Data (OOP)

Extract common parts of two different data types and put them in a super-class. That's **Inheritance!** 🏠

## Chapter 9

# Taxonomy: Abstraction of Abstractions



Taxonomy is the ultimate goal of the abstraction process — organizing knowledge into ordered, hierarchical systems. Think of it as abstraction leveling up. 

# Successful Taxonomies in the Wild



## Linguistic

Classification of languages into families, branches, and dialects.



## Chemistry

The periodic table of elements — the most famous taxonomy in science.



## Biology

Kingdom → Phylum → Class → Order → Family → Genus → Species.  
The OG hierarchy.



## Mathematics

Algebraic structures: group → ring → commutative-ring → field → algebra...

# Lingual Abstraction Mechanisms

01

---

## Procedure Call

As in structured programming — the classic building block.

ćč

---

## Modules

As in modular programming — group related things together.

03

---

## Abstract Data Types

Export type definition + operations. Protect data. Instantiation. Object-based programming.

04

---

## Inheritance

Classification of ADTs. Similarity between abstract data types.

ćđ

---

## Dynamic Binding & Genericity

Actual representation independence. Similarity between operations on different types.

# Object-Oriented Analysis (OOA)

What is OOA?

Given a problem domain, decide on the correct set of abstractions.

Simple in theory. Fiendishly hard in practice. 🤔

Activities

- Identify classes (and "objects"?) in the problem domain
- Discover classes
- Invent generalizations (Abstract classes)

OOA methods are recipes, heuristics, and techniques to help you abstract over the abstraction process itself.

## Chapter 10

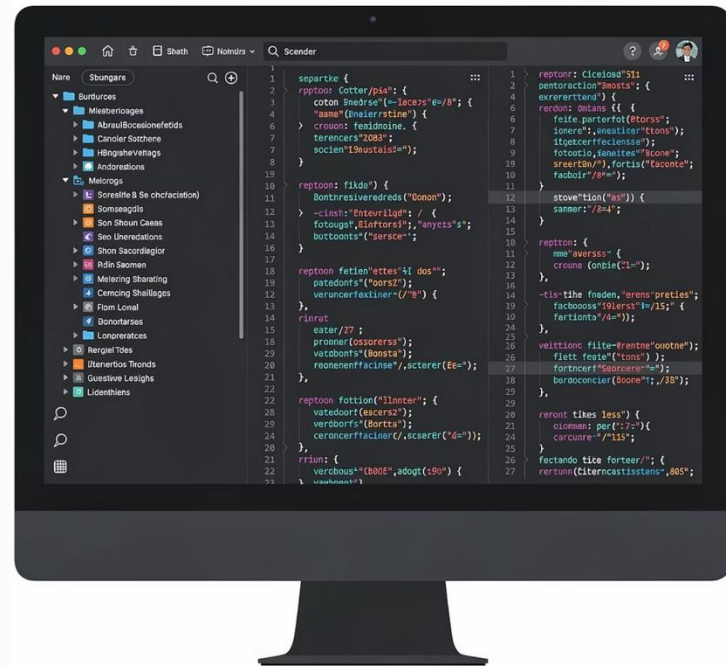
# Real Techniques: Language Level

### Abstraction Mechanisms

The foundation — the language-level tools that make everything else possible.

### Evangelist Level

Style guidelines and rules. The "thou shalt not" of good coding. 📖



### ACTERICHE

Betes /http://erst-rotstten/  
Gest applg lixolouns, deriy-phding  
witing the soves-trebbder  
bouds the frener!

# Real Techniques: Code Level



## Static Strong Typing

Subtypes, polymorphism, const correctness, genericity. A program will never make a type error.



## Testing & Code Metrics

Measure everything. If you can't measure it, you can't improve it.



## Formal Verification

Mathematical proof that your code does what it claims. The gold standard. 🏆



## Real Techniques: Run Time Level

### Garbage Collection

A program will never access an invalid address in memory, nor will it create memory leaks. Your memory is safe. (Finally.)

### Assertions

Pre-conditions, post-conditions, loop invariants, loop variants, class invariants, checkpoints. A program will not proceed if it violates certain design decisions. The bouncer of your codebase. 

## Chapter 11

# Teaching Design: What We Can and Can't Do

### We CANNOT...

Teach you how to design. Make you smart.  
Force you to think. Make you paranoid.  
Make you pedantic. Remake you. 🤖

### We CAN Teach...

Good designs made by others. Tricks of the trade. Trivial rules. (Trivial rules are still rules!)

### We Can Hope...

That you will grow to be a good designer.  
🌱 That's the dream.



## We Can Force You To...

### Practice Design

The only way to get better is to do it. Repeatedly. Even when it hurts.

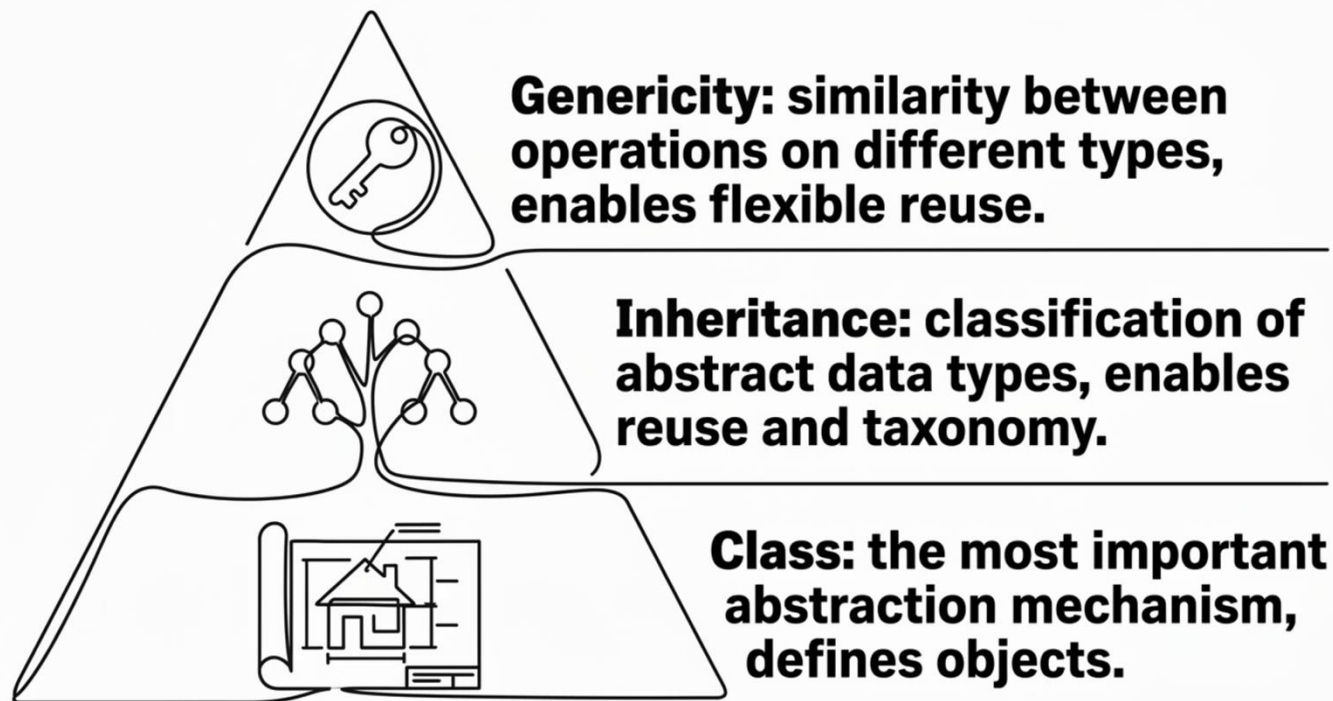
### Abide by Our Subjective Criteria

Yes, it's subjective. Yes, it still counts. Welcome to software engineering. 🎓



The goal: grow into a designer who makes good choices instinctively, not just when forced to.

# The Three Pillars of OOP



Remember: OOP is about **classes**, not objects. Classes are the abstraction. Objects are just instances. Don't let anyone tell you otherwise. 🙄

# The Software Eras: A Quick Recap

1

First Era (1950s)

Batch Oriented, Limited Distribution, Custom Software

2

Second Era (1960s)

Multuser, Real-Time, Database, Product Software

3

Third Era (1970s)

Distributed Systems, Low-Cost Hardware, Consumer Impact

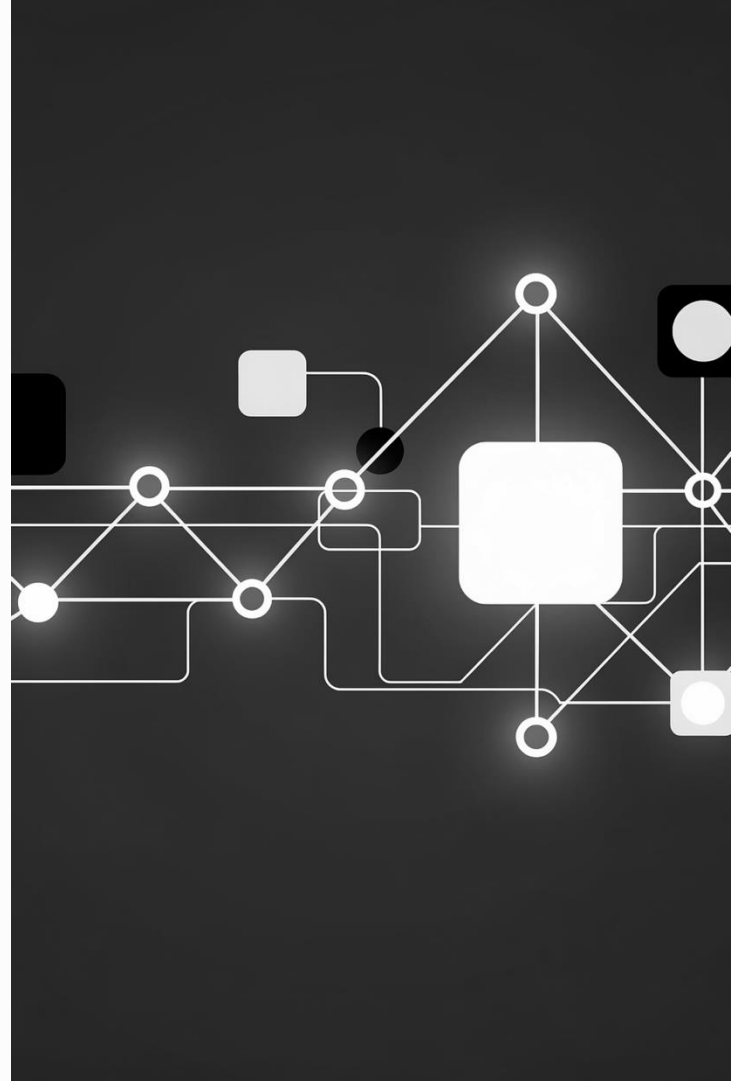
4

Fourth Era (1980s+)

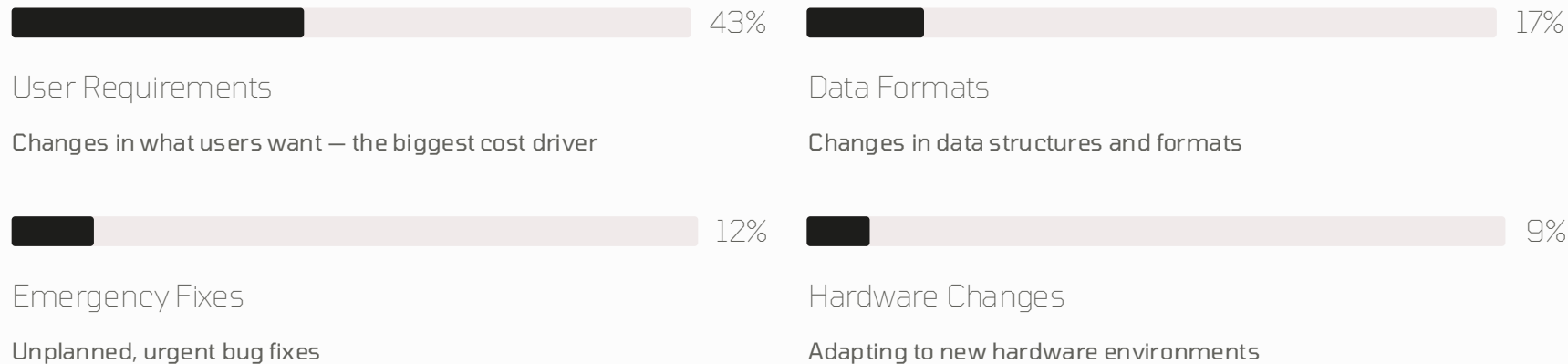
OOP, Expert Systems, Neural Nets, Parallel Computing

# OOP's Role in the Fourth Era

The explosive growth of computer speeds and capabilities at very low cost fueled demand for very complex software and increased customer expectations. OOP emerged as a key technique to manage this complexity — not a silver bullet, but a powerful tool in the right hands. 🛠️

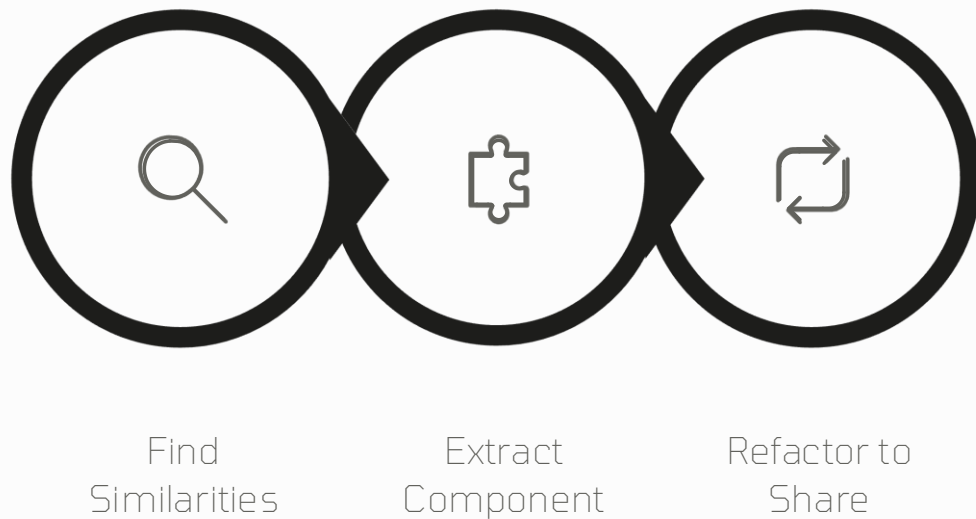


# The Maintenance Breakdown: What Costs What



Source: Lientz & Swanson '79. The numbers are old. The pain is timeless. 😂

# Abstraction: The Reuse Engine



Abstraction is the engine that drives reusability. Find what's common, extract it, and share it. This is the core loop of good software design — and it never gets old.

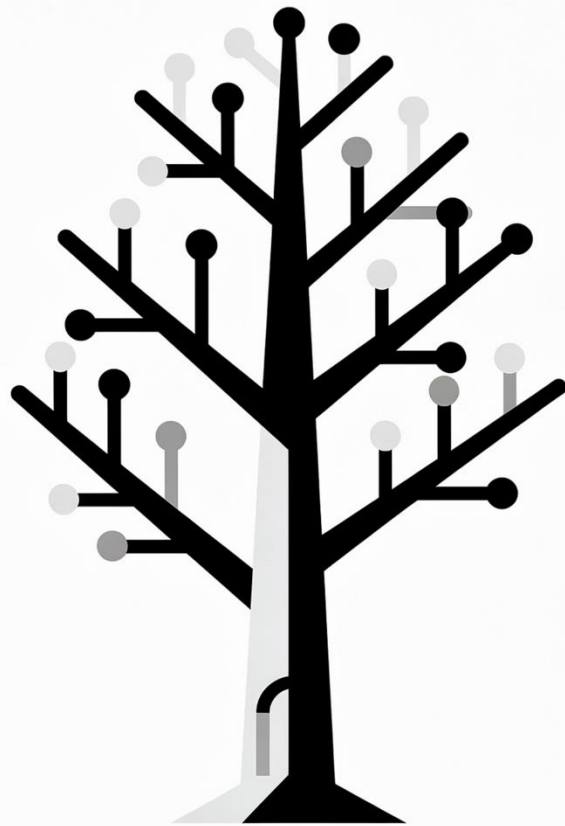
# Inheritance: The Taxonomy of Code

## What Inheritance Does

Extracts common parts of two different data types and places them in a super-class. Reduces redundancy. Enables taxonomy. Makes your code a well-organized library, not a junk drawer. 📚

## Real-World Analogy

Biology: Kingdom → Phylum → Class → Order → Family → Genus → Species. Inheritance in OOP works the same way — hierarchy with shared traits flowing downward.



# Dynamic Binding & Genericity

1

## Dynamic Binding

Actual representation independence. The program decides at runtime which version of an operation to call. Flexibility at its finest. 🏆

2

## Genericity

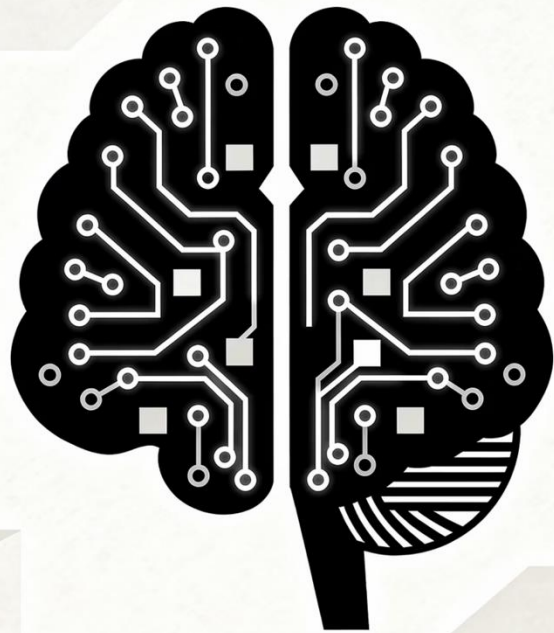
Similarity between operations on different types. Write once, use with any type. The ultimate reuse mechanism.

📘 Together, these mechanisms give OOP its power to model the real world flexibly and efficiently.

# The Complexity Challenge: Human Ingenuity

Software is just like mathematical theorems: good notations, adequate environments, and powerful tools help — but the main factor remains **human Ingenuity**.

No tool, paradigm, or methodology replaces a smart, thoughtful engineer. OOP gives you better tools. You still have to wield them well. 🧠



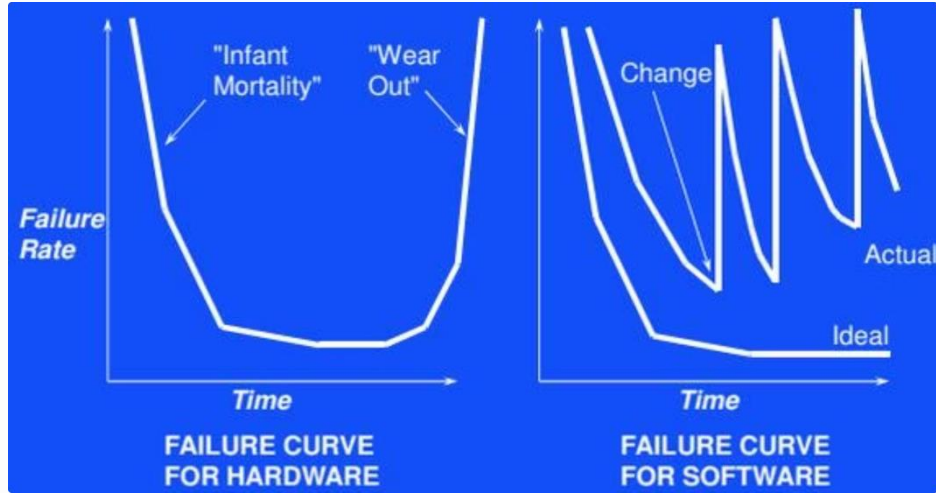
# The "1 Child = 9 Months" Problem

Adding programmers to a late software project makes it later. 1 child = 9 months, no matter how many women are assigned. — Fred Brooks (paraphrased)

OOP has the potential for more disciplined interconnectivity — but only if used carefully. Undisciplined OOP can make things worse. 🤔



# The Failure Curve: Hardware vs. Software



## Hardware

U-shaped curve: infant mortality → stable period → wear out.  
Predictable and manageable.

## Software

Ideal: flat line. Actual: chaotic spikes every time something changes. Change is the enemy of stability. 📈

# OOP: The Honest Summary



## What OOP Does

Provides better abstraction, reusability, extendibility, and a more realistic model of the world.



## What OOP Doesn't Do

Make bad programmers good. Guarantee success. Solve all software engineering problems.



## The Real Goal

Quality software. When quality is pursued, productivity follows — K. Fujino, NEC.

# Design Principles to Live By



## 0 > 1: Information Hiding

What you don't know cannot hurt you. Encapsulate everything.



## Reuse First

Build from existing parts. Identify similarity. Create abstractions.



## 1 > 2: No Duplication

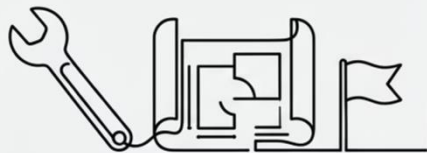
If it appears twice, it appears once too many. Extract and share.



## Quality Over Speed

There's never time to do it right, but always time to do it over.  
Choose wisely.

# What We Learned (And What We Hope You Did Too)



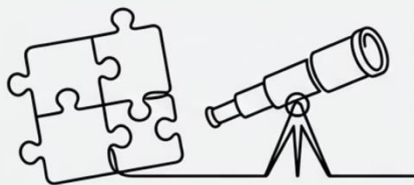
**OOP IS AN  
ENGINEERING  
TECHNIQUE**



**CLASSES ARE THE  
CORE ABSTRACTION**



**QUALITY LEADS  
TO PRODUCTIVITY**



**REUSABILITY &  
EXTENDIBILITY  
ARE MAIN BENEFITS**

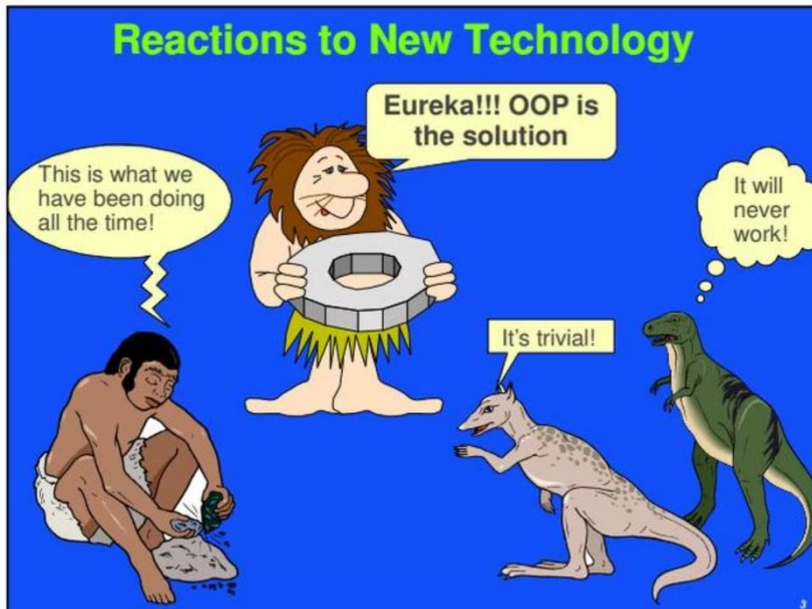


**OOP HAS  
OOP HAS REAL LIMITS  
(NO MIRACLES)**



**HUMAN INGENUITY  
IS THE MAIN FACTOR**

# The Four Reactions – Which Are You Now?



 Caveman

"This is what we've been doing all the time!"

 Skeptic

"It's trivial!"

 Enthusiast

"Eureka! OOP is the solution!"

 Pessimist

"It will never work!"



# Go Forth and Design Well!

*"When quality is pursued, productivity will follow."*

— K. Fujino, NEC Corporation of Japan

We can't make you a great designer. We can't force you to think. But we can hope — and we do — that you'll grow into one. Now go write some beautiful, reusable, extendible, non-redundant code. 🚀



Remember: OOP is about **classes**, not objects. Don't let anyone tell you otherwise.